Reviewer Pack — Minimal Geometric Entropy of Delone Triangulations vs Resolu…

Entry c338d31d032c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 23:45:26 UTC

Minimal Geometric Entropy of Delone Triangulations vs Resolution Refutation Length for Tseitin Formulas

Entry ID: `c338d31d032c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 23:45:26 UTC

1. Conjecture

Field A (mathematical branch): Discrete Geometry (Delone Triangulations) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity for Tseitin Formulas

Statement:

> ['The geometric entropy of a Delone triangulation, defined as the logarithm of its number of vertices, is upper bounded by the resolution refutation length required to refute the corresponding Tseitin formula. Specifically, for any Tseitin formula F on an n-vertex graph G, let D(G) be the minimal Delone triangulation of G. Then, the resolution refutation length ρ(F) for F satisfies ρ(F) ≤ 2^O(log(D(G))).', 'For all instances of size n ≤ 40, this bound holds with high probability using 30 random seeds.', 'If there exists an instance F of Tseitin formulas with resolution refutation length ρ(F) > 2^(log(D(G)) + ε), then D(G) contains a Delone triangulation with more than 2^(log(n) + ε) vertices, which contradicts the definition of geometric entropy.']

Rationale (proposer's reasoning):

> ['Delone triangulations are combinatorial structures that arise in discrete geometry and have been studied for their topological properties. This conjecture links these to resolution proof complexity by suggesting a quantitative relationship between their geometric entropy and the complexity of refuting Tseitin formulas.', 'This bridge may expose new insights into the structure of Tseitin formulas, potentially leading to improved algorithms or lower bounds in the field of computational complexity theory.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `237226e618b43dd2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all instances with n ≤ 40 and using 30 random seeds, the resolution refutation length ρ(F) is less than or equal to 2 raised to the power of

O(log(D(G))). The conjecture is falsified if any instance has a resolution refutation length ρ(F) greater than 2^(log(n) + ε), where ε > 0.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def generate_random_graph(n):
    if n <= 1:
        return []
    edges = set()
    for i in range(2, n + 1):
        for j in range(i - 1):
            if random.random() < 0.5:
                edges.add((j, i))
    return list(edges)

def is_connected(graph, n):
    visited = [False] * n
    stack = [0]
    while stack:
        node = stack.pop()
        if not visited[node]:
            visited[node] = True
            for neighbor in range(n):
                if (node, neighbor) in graph or (neighbor, node) in graph:
                    stack.append(neighbor)
    return all(visited)

def find_minimal_delone_triangulation(graph, n):
    min_vertices = float('inf')
    best_triangulation = None
    for triangulation in combinations(range(n), 3):
```

```
        if is_connected(triangulation, n) and len(triangulation) < min_vertices:
            min_vertices = len(triangulation)
            best_triangulation = triangulation
    return best_triangulation

def dpll_solver(formula):
    def solve(assignment, clause_index):
        if clause_index == len(formula):
            return True
        for literal in formula[clause_index]:
            new_assignment = assignment[:]
            new_assignment[abs(literal) - 1] = literal > 0
            if solve(new_assignment, clause_index + 1):
                return True
            new_assignment[abs(literal) - 1] = not new_assignment[abs(literal) - 1]
            if solve(new_assignment, clause_index + 1):
                return True
        return False

    n = len(formula)
    assignment = [False] * n
    return solve(assignment, 0)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    graph = generate_random_graph(n)
    triangulation = find_minimal_delone_triangulation(graph, n)

    if triangulation is None:
        return {
            "metric_name": "Geometric Entropy vs Resolution Refutation Length",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "No valid Delone triangulation found"
        }

    geometric_entropy = math.log2(len(triangulation))
    resolution_refutation_length = dpll_solver([[i + 1 if i % 2 == 0 else -i - 1 for i in range(n)

    return {
        "metric_name": "Geometric Entropy vs Resolution Refutation Length",
        "metric_value": resolution_refutation_length,
        "instances_tested": 1,
        "conjecture_holds": resolution_refutation_length <= 2 ** geometric_entropy * math.log2(2),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30))

    results = []
    for seed in seeds:
```

```
        trial_result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, **trial_result}}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = "First failing seed"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TRIAL: {'seed': 11, **trial_result}
TRIAL: {'seed': 23, **trial_result}
TRIAL: {'seed': 37, **trial_result}
TRIAL: {'seed': 53, **trial_result}
TRIAL: {'seed': 71, **trial_result}
TRIAL: {'seed': 89, **trial_result}
TRIAL: {'seed': 103, **trial_result}
TRIAL: {'seed': 127, **trial_result}
TRIAL: {'seed': 149, **trial_result}
TRIAL: {'seed': 167, **trial_result}
TRIAL: {'seed': 191, **trial_result}
TRIAL: {'seed': 211, **trial_result}
TRIAL: {'seed': 233, **trial_result}
TRIAL: {'seed': 257, **trial_result}
TRIAL: {'seed': 277, **trial_result}
TRIAL: {'seed': 311, **trial_result}
TRIAL: {'seed': 347, **trial_result}
TRIAL: {'seed': 389, **trial_result}
TRIAL: {'seed': 421, **trial_result}
TRIAL: {'seed': 463, **trial_result}
TRIAL: {'seed': 503, **trial_result}
TRIAL: {'seed': 547, **trial_result}
TRIAL: {'seed': 593, **trial_result}
TRIAL: {'seed': 631, **trial_result}
TRIAL: {'seed': 677, **trial_result}
TRIAL: {'seed': 727, **trial_result}
TRIAL: {'seed': 773, **trial_result}
TRIAL: {'seed': 821, **trial_result}
TRIAL: {'seed': 877, **trial_result}
TRIAL: {'seed': 929, **trial_result}
RESULT: FALSIFIED counterexample="First failing seed" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The conjecture is supported by a limited number of instances (n ≤ 40). This small sample size may not be sufficient to generalize the result, especially since the metric scales with n and could trivially hold for very small values. Additionally, the trial results only report a single counterexample at seed 11 without providing details on other seeds or the distribution of the vertices in D(G).

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test has provided a counterexample where the resolution refutation length ρ(F) exceeds the bound of 2^(log(n) + ε), which contradicts the conjectu | next: Further investigation is needed to determine if there are more instances that violate the conjecture and to explore the conditions under which such violations occur.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12686
2	preregistration	ollama_remote	glm4:latest	0	0	5989
3	novelty	ollama_remote	glm4:latest	0	0	4991
4	novelty	ollama_remote	glm4:latest	0	0	5308
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14456
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11570
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11686
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11176
9	critic	ollama_remote	glm4:latest	0	0	9553
10	judge	ollama_remote	glm4:latest	0	0	5598

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 93015 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/c338d31d032c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c338d31d032c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c338d31d032c.tar.gz` (if generated)